Building a Custom Transformer for NMT at Scale

Topic: Engineering & Mathematical Implementation of a Sequence-to-Sequence Transformer from scratch using TensorFlow and Keras, optimized for massive datasets (e.g., 22.5M rows English-French corpus) within resource-constrained environments like Kaggle Notebooks.

1. Strategic Roadmap for "Big Data" NLP

When handling datasets containing tens of millions of sentence pairs (spanning multiple gigabytes), traditional preprocessing pipelines often trigger Out-Of-Memory (OOM) errors. The core engineering strategy relies on two tenets:

- **Subsampling during Pipeline Prototyping:** Isolate a functional chunk (e.g., 100,000 to 500,000 rows) to rapidly build, test, and mathematically verify the neural network before scaling to the entire dataset.
- **Memory-Mapped Datasets & Streaming:** Avoid loading raw CSV strings into standard memory via tools like Pandas. Instead, leverage generator-based parsing or native `tf.data.Dataset` pipelines that lazily load data sequences from disk on demand.

Model Architecture Options

Model Choice	Pros	Cons	Best Used For
MarianMT (Helsinki-NLP)	Lightweight, fast, pre-trained specifically for translation.	Less generalized than large LLMs.	Production translation on a limited compute budget.
T5 (small / base)	Superb transfer learning and cross-task generalization.	Requires prefix text formatting and more VRAM.	Benchmark testing on standard hardware.
Custom Transformer	Complete architectural control; highly educational.	Massive compute footprint; long training times from scratch.	Custom syntax handling and studying deep learning mechanics.

2. Data Preparation Pipeline (TensorFlow/Keras)

To avoid memory overhead, the text files are loaded iteratively. We wrap the French target sequences with explicit structural boundaries: `[start]` and `[end]` tokens. These are used during the text vectorization layer to organize the temporal shifts required for autoregressive processing.

```
import csv
import itertools
import tensorflow as tf
from tensorflow.keras.layers import TextVectorization
from sklearn.model_selection import train_test_split

# Path configuration in Kaggle environments
csv_path = "/kaggle/input/en-fr-translation-dataset/en-fr.csv"
SUBSET_SIZE = 500_000

en_sentences = []
fr_sentences = []

with open(csv_path, mode='r', encoding='utf-8') as f:
    reader = csv.DictReader(f)
    for row in itertools.islice(reader, SUBSET_SIZE):
        if row['en'] and row['fr']:
            en_sentences.append(row['en'].strip())
            fr_sentences.append(f"[start] {row['fr'].strip()} [end]")

# Train/Validation Partitioning
train_en, val_en, train_fr, val_fr = train_test_split(
    en_sentences, fr_sentences, test_size=0.1, random_state=42
)
```

Tokenization via Text Vectorization

The `TextVectorization` layer automatically builds internal index mapping tables, handles punctuation stripping, standardizes casing, and structures dynamic sequences on the CPU/GPU background threads.

```
VOCAB_SIZE = 20000
MAX_SEQUENCE_LENGTH = 64

# English (Source) Vectorizer
en_vectorizer = TextVectorization(
    max_tokens=VOCAB_SIZE,
    output_mode="int",
    output_sequence_length=MAX_SEQUENCE_LENGTH,
)

# French (Target) Vectorizer - preserve structural boundary brackets
fr_vectorizer = TextVectorization(
    max_tokens=VOCAB_SIZE,
    output_mode="int",
    output_sequence_length=MAX_SEQUENCE_LENGTH + 1,
    standardize=lambda text: tf.strings.lower(tf.strings.regex_replace(text,
r"[[:punct:]]", ""))
)

# Adapt maps to build dictionaries from training corpora
en_vectorizer.adapt(train_en)
fr_vectorizer.adapt(train_fr)
```

3. Mathematical Structuring: Teacher Forcing Shifts

In a Sequence-to-Sequence Transformer, the Decoder receives target sequences shifted by exactly one position during training. This training strategy is known as **Teacher Forcing**. The model takes two explicit matrix inputs:

1. **Encoder Input:** The complete source sequence (English tokens).
2. **Decoder Input:** The target sequence (French tokens) containing the `[start]` token, but discarding the trailing `[end]` token.

The objective of the model is to calculate a probability distribution matching the **Decoder Target Output Labels**, which contain the target sequence discarding the leading `[start]` token but keeping the trailing `[end]` token.

```
def format_dataset(eng, fre):
    eng_tok = en_vectorizer(eng)
    fre_tok = fr_vectorizer(fre)

    # Shift input sequences by slicing
    dec_input = fre_tok[:, :-1]  # Drop the final token
    dec_target = fre_tok[:, 1:]   # Drop the leading [start] token

    return ({"encoder_inputs": eng_tok, "decoder_inputs": dec_input}, dec_target)

# Optimized tf.data Input Pipelines
BATCH_SIZE = 64

def make_dataset(eng_sentences, fre_sentences, batch_size=BATCH_SIZE, is_train=True):
    dataset = tf.data.Dataset.from_tensor_slices((eng_sentences, fre_sentences))
    if is_train:
        dataset = dataset.shuffle(2048)
    dataset = dataset.batch(batch_size)
    dataset = dataset.map(format_dataset, num_parallel_calls=tf.data.AUTOTUNE)
    return dataset.prefetch(tf.data.AUTOTUNE)

train_ds = make_dataset(train_en, train_fr, is_train=True)
val_ds = make_dataset(val_en, val_fr, is_train=False)
```

4. Embedding and Positional Encoding

Because Transformers use multi-head attention blocks instead of sequential loops (like RNNs/LSTMs), they process all words simultaneously and have no inherent concept of word order. Positional information must be explicitly added to token vectors. The standard Transformer architecture implements deterministic sinusoidal wave frequencies:

$$PE_{(pos,\ 2i)} = sin(pos\ /\ 10000^{2i\ /\ d_{model}})$$

$$PE_{(pos,\ 2i+1)} = cos(pos\ /\ 10000^{2i\ /\ d_{model}})$$

Below is the custom implementation combining standard word token embedding with static positional encoding matrices inside a reusable Keras layer:

```
import numpy as np

class TransformerEmbedding(layers.Layer):
    def __init__(self, vocab_size, embed_dim, max_length, **kwargs):
        super().__init__(**kwargs)
        self.vocab_size = vocab_size
        self.embed_dim = embed_dim
        self.max_length = max_length

        self.token_embeddings = layers.Embedding(
            input_dim=vocab_size, output_dim=embed_dim, mask_zero=True
        )
        self.pos_encodings = self._get_positional_encoding(max_length, embed_dim)

    def _get_positional_encoding(self, max_length, embed_dim):
        pos_enc = np.zeros((max_length, embed_dim))
        positions = np.arange(max_length)[:, np.newaxis]
        dims = np.arange(embed_dim)[np.newaxis, :]

        angle_rates = 1 / np.power(10000, (2 * (dims // 2)) / np.float32(embed_dim))
        angle_radians = positions * angle_rates

        pos_enc[:, 0::2] = np.sin(angle_radians[:, 0::2])
        pos_enc[:, 1::2] = np.cos(angle_radians[:, 1::2])

        return tf.cast(pos_enc[np.newaxis, ...], dtype=tf.float32)

    def call(self, inputs):
        x = self.token_embeddings(inputs)
        # Scale embeddings by square root of embedding dimensions
        x *= tf.math.sqrt(tf.cast(self.embed_dim, tf.float32))
        length = tf.shape(inputs)[-1]
        x += self.pos_encodings[:, :length, :]
        return x

    def compute_mask(self, inputs, mask=None):
        return self.token_embeddings.compute_mask(inputs, mask)
```

5. The Multi-Head Attention Substructures

Using Keras's optimized `layers.MultiHeadAttention` ensures high-speed tensor compilation. The network maps out two core types of attention blocks:

- **Encoder Blocks:** Leverage *Self-Attention* where queries, keys, and values are sourced entirely from the input sentence.

- **Decoder Blocks:** Run a two-stage attention process: a *Causal Masked Self-Attention* block to preserve autoregressive token constraints, followed by a *Cross-Attention* block mapping Decoder structures back into Encoder output sequences.

Encoder Block Subclass

```
class TransformerEncoderBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1, **kwargs):
        super().__init__(**kwargs)
        self.mha = layers.MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)
        self.ffn = tf.keras.Sequential([
            layers.Dense(ff_dim, activation="relu"),
            layers.Dense(embed_dim)
        ])
        self.layernorm1 = layers.LayerNormalization(epsilon=1e-6)
        self.layernorm2 = layers.LayerNormalization(epsilon=1e-6)
        self.dropout1 = layers.Dropout(rate)
        self.dropout2 = layers.Dropout(rate)

    def call(self, inputs, training=False):
        attn_output = self.mha(query=inputs, value=inputs, key=inputs)
        out1 = self.layernorm1(inputs + self.dropout1(attn_output, training=training))
        ffn_output = self.ffn(out1)
        return self.layernorm2(out1 + self.dropout2(ffn_output, training=training))
```

Decoder Block Subclass

```
class TransformerDecoderBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1, **kwargs):
        super().__init__(**kwargs)
        self.causal_mha = layers.MultiHeadAttention(num_heads=num_heads,
key_dim=embed_dim)
        self.cross_mha = layers.MultiHeadAttention(num_heads=num_heads,
key_dim=embed_dim)
        self.ffn = tf.keras.Sequential([
            layers.Dense(ff_dim, activation="relu"),
            layers.Dense(embed_dim)
        ])
        self.layernorm1 = layers.LayerNormalization(epsilon=1e-6)
        self.layernorm2 = layers.LayerNormalization(epsilon=1e-6)
        self.layernorm3 = layers.LayerNormalization(epsilon=1e-6)
        self.dropout1 = layers.Dropout(rate)
        self.dropout2 = layers.Dropout(rate)
        self.dropout3 = layers.Dropout(rate)

    def call(self, inputs, encoder_outputs, training=False):
        # 1. Causal masked attention (restricts look-ahead visibility)
        causal_attn = self.causal_mha(query=inputs, value=inputs, key=inputs,
use_causal_mask=True)
        out1 = self.layernorm1(inputs + self.dropout1(causal_attn, training=training))

        # 2. Cross-attention to Encoder features
        cross_attn = self.cross_mha(query=out1, value=encoder_outputs,
key=encoder_outputs)
        out2 = self.layernorm2(out1 + self.dropout2(cross_attn, training=training))

        # 3. Position-wise feed-forward
        ffn_output = self.ffn(out2)
        return self.layernorm3(out2 + self.dropout3(ffn_output, training=training))
```

6. Complete Model Composition & Optimization

Using the Keras Functional paradigm, the structural layers are interconnected within an explicit model framework. We iterate blocks dynamically based on the `num_layers` configuration value.

```
class FullTransformer(tf.keras.Model):
    def __init__(self, num_layers, embed_dim, num_heads, ff_dim,
                 src_vocab_size, trg_vocab_size, max_len, rate=0.1, **kwargs):
        super().__init__(**kwargs)
        self.encoder_embedding = TransformerEmbedding(src_vocab_size, embed_dim,
max_len)
        self.decoder_embedding = TransformerEmbedding(trg_vocab_size, embed_dim,
max_len)

        self.encoder_layers = [TransformerEncoderBlock(embed_dim, num_heads, ff_dim,
rate) for _ in range(num_layers)]
        self.decoder_layers = [TransformerDecoderBlock(embed_dim, num_heads, ff_dim,
rate) for _ in range(num_layers)]
        self.final_layer = layers.Dense(trg_vocab_size, activation="softmax")

    def call(self, inputs, training=False):
        encoder_inputs = inputs["encoder_inputs"]
        decoder_inputs = inputs["decoder_inputs"]

        # Process through full Encoder stack
        enc_x = self.encoder_embedding(encoder_inputs)
        for encoder_layer in self.encoder_layers:
            enc_x = encoder_layer(enc_x, training=training)

        # Process through full Decoder stack
        dec_x = self.decoder_embedding(decoder_inputs)
        for decoder_layer in self.decoder_layers:
            dec_x = decoder_layer(dec_x, encoder_outputs=enc_x, training=training)

        return self.final_layer(dec_x)
```

Optimization Note: Masked Loss Calculation

Because padding tokens (0) are heavily prevalent in sequences, standard sparse loss functions will miscalculate gradients by prioritizing padding optimization. We mask out 0 indices explicitly to calculate the true gradient updates.

```
def masked_sparse_categorical_crossentropy(y_true, y_pred):
    loss_object = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False,
reduction='none')
    loss = loss_object(y_true, y_pred)
    mask = tf.cast(tf.math.not_equal(y_true, 0), dtype=loss.dtype)
    loss *= mask
    return tf.reduce_sum(loss) / tf.reduce_sum(mask)

# Compilation Setup
NUM_LAYERS = 2
EMBED_DIM = 256
NUM_HEADS = 4
FF_DIM = 512

transformer_model = FullTransformer(NUM_LAYERS, EMBED_DIM, NUM_HEADS, FF_DIM,
VOCAB_SIZE, VOCAB_SIZE, MAX_SEQUENCE_LENGTH)
transformer_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss=masked_sparse_categorical_crossentropy,
    metrics=["accuracy"]
)
```

7. Autoregressive Inference (Translation Decoding)

Transformers generate translations step-by-step. The `translate_sentence` utility function feeds previous predictions recursively back into the input sequence array until the terminating boundary token is encountered.

```
def translate_sentence(sentence, model, en_vectorizer, fr_vectorizer, max_len=64):
    enc_input = en_vectorizer([sentence])
    start_index = fr_vectorizer.get_vocabulary().index("start")
    end_index = fr_vectorizer.get_vocabulary().index("end")

    dec_input = tf.expand_dims([start_index], 0)

    for i in range(max_len):
        dec_input_padded = tf.keras.preprocessing.sequence.pad_sequences(dec_input,
maxlen=max_len + 1, padding='post')
        predictions = model({"encoder_inputs": enc_input, "decoder_inputs":
dec_input_padded}, training=False)
        predicted_id = tf.argmax(predictions[0, i, :]).numpy()
        dec_input = tf.concat([dec_input, [[predicted_id]]], axis=-1)

        if predicted_id == end_index:
            break

    vocab = fr_vectorizer.get_vocabulary()
    words = [vocab[idx] for idx in dec_input[0].numpy() if idx not in (start_index,
end_index, 0)]
    return " ".join(words)
```